// DEEP DIVE

Detection Engineering

FIG_000 · guide v1.0 · 2026 · open reference · 9 sections

by Swastik Sagar

Writing a single correlation rule is easy. Building, testing, deploying, and measuring detections as a real engineering discipline — with version control, staging, and honest metrics — is the actual job.

```
$ read section_01 --start
```

WHAT'S INSIDE

SECTION	TOPIC
1 — 2	What detection engineering is, and its full lifecycle
3 — 5	Writing detections, testing before deployment, and detection-as-code
6 — 9	Measuring efficacy, ATT&CK coverage mapping, a worked example, and reference

How to use this guide: read start to finish for the full picture, or jump to Section 8 for the worked build-and-test example. Every diagram is numbered and referenced directly in the surrounding text.

// INDEX

Table of Contents

This guide is designed to be read section by section, with diagrams positioned next to the concepts they illustrate.

// SECTION 1

What Detection Engineering Actually Is

1.1 MORE THAN WRITING ONE RULE

The SIEM & Log Analysis guide's Section 8 walked through building a single correlation rule, layer by layer. **Detection engineering** is the broader discipline that surrounds that one rule: deciding what's worth detecting in the first place, writing it in a testable, version-controlled way, validating it actually works before it reaches production, and continuously measuring whether it's still earning its place months later.

1.2 WHY THIS NEEDS TO BE ITS OWN DISCIPLINE

Treating detections as one-off, hand-written artifacts — created once when a need arises, then left alone indefinitely — is exactly how a SIEM ends up with hundreds of stale, untested, or silently-broken rules (recall the SIEM & Log Analysis guide's Section 6.3 point about silent parsing failures). Detection engineering applies the same rigor software engineering applies to code — version control, testing, staged rollout, ongoing measurement — to the specific problem of building durable detections.

> **A detection is a hypothesis, not a fact.** Every rule encodes a belief: "this pattern indicates malicious activity." Like any hypothesis, it needs to be testable (Section 4), it can be wrong (Section 6's false positive/negative tracking), and it needs revisiting as the environment and threat landscape change — treating a deployed rule as permanently correct is the single most common failure mode this discipline exists to prevent.

1.3 WHERE THIS FITS AMONG THE OTHER GUIDES

This guide sits directly between two others: the Threat Intelligence Fundamentals guide supplies *what's worth detecting* (via the Pyramid of Pain and ATT&CK techniques, Sections 3–4 of that guide), and the SIEM & Log Analysis guide supplies the *mechanics* of how a correlation rule actually works. Detection engineering is the process that connects them — turning "here's a technique worth watching for" into a tested, measured, production detection.

// SECTION 2

The Detection Engineering Lifecycle

2.1 A REPEATING, SIX-STAGE PROCESS

Like the IR lifecycle (Incident Response Playbook Companion guide, Section 1) and the intelligence cycle (Threat Intelligence Fundamentals guide, Section 6), detection engineering follows a structured, repeating loop rather than a one-time linear task.

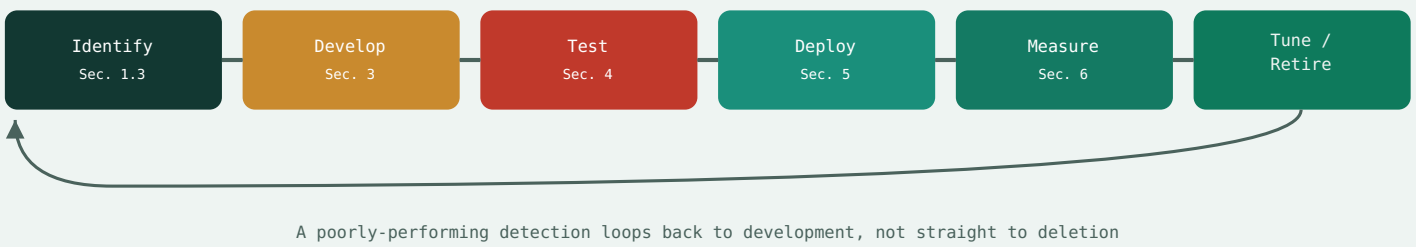

Fig 2.1 — The detection engineering lifecycle: each stage maps to a later section of this guide, with tuning/retirement feeding back into development rather than the cycle simply ending.

2.2 STAGE SUMMARY

STAGE	QUESTION IT ANSWERS
Identify	What technique or behavior is worth detecting? (Threat Intelligence Fundamentals guide, Sections 3–4)
Develop	What's the actual logic that catches it? (Section 3)
Test	Does it work, and how noisy is it, before real users see it? (Section 4)
Deploy	How does it reach production safely? (Section 5)
Measure	Is it actually earning its place? (Section 6)
Tune / Retire	Adjust it, or remove it if it's no longer valuable — feeding back into Develop

// SECTION 3

Writing Detections: From Idea to Rule

3.1 START FROM BEHAVIOR, NOT FROM A LOG FIELD

The strongest detections start from a specific, documented technique — an ATT&CK technique ID (Threat Intelligence Fundamentals guide, Section 4.2), a real intelligence report, or a confirmed incident's root cause — rather than starting from "what fields does this log source happen to have" and reverse-engineering a rule to fit. This is the IOA-first philosophy from that same guide's Section 2.3, applied as a starting discipline rather than an afterthought.

3.2 THE DETECTION REQUIREMENTS CHECKLIST

QUESTION	WHY IT MATTERS
What specific behavior am I detecting?	Vague goals produce vague, unreliable rules
What log source(s) actually capture this?	A detection can't fire on data that isn't collected — verify the source exists and is reliably logging (SIEM & Log Analysis guide, Section 1.2's audit-policy point)
What does legitimate activity matching this pattern look like?	Understanding expected false positives up front informs both logic and thresholds
What severity does a real match actually warrant?	Sets the alert priority and expected response urgency

3.3 SPECIFICITY, REVISITED

The SIEM & Log Analysis guide's Section 5.3 introduced rule specificity generically; the practical version of that advice is: prefer matching the *combination* of conditions that's actually distinctive (a specific process, a specific argument pattern, a specific parent-child relationship) over any single condition alone, which is usually too broad on its own to be a reliable signal.

```
$ # Too broad — matches enormous amounts of legitimate activity
$ Image|endswith: '\powershell.exe'

$ # Specific — the combination is what's actually distinctive
$ Image|endswith: '\powershell.exe'
$ CommandLine|contains: '-enc'
$ ParentImage|endswith: '\winword.exe'
```

The second version — PowerShell, launched with an encoded command, spawned specifically from Word — is the same kind of parent-child-plus-argument specificity already demonstrated in the Windows Event Log Reference guide's Section 4.2 and the Malware Analysis Basics guide's Section 6.2, now framed as a deliberate writing principle rather than a one-off example.

// SECTION 4

Testing Detections Before Deployment

4.1 WHY "IT LOOKS RIGHT" ISN'T ENOUGH

A detection rule that appears logically correct can still fail in production for reasons invisible from just reading the logic — a field name that's slightly different in real data, a log source that doesn't actually populate the field you're matching on, or a false-positive rate so high the rule is useless the moment it's live. Testing exists to catch these before an analyst, not a test suite, discovers them.

4.2 TWO KINDS OF TESTING

TEST TYPE	VALIDATES	HOW
Positive testing	The rule actually fires on the behavior it's meant to catch	Generate the real behavior in a safe test environment (see the Malware Analysis Basics guide's sandbox, Section 2) and confirm detection
Negative testing	The rule doesn't fire on similar-but-legitimate activity	Run the rule against known-good historical data and confirm no unexpected matches

4.3 STAGED ROLLOUT

Even after passing both test types, deploying a new detection directly to full, alert-generating production is risky — instead, most mature detection programs run a new rule in a **silent/audit mode** first, logging what it *would have* alerted on without actually paging anyone, over a real production data volume for a meaningful period (days to weeks) before promoting it to a live, alerting rule.

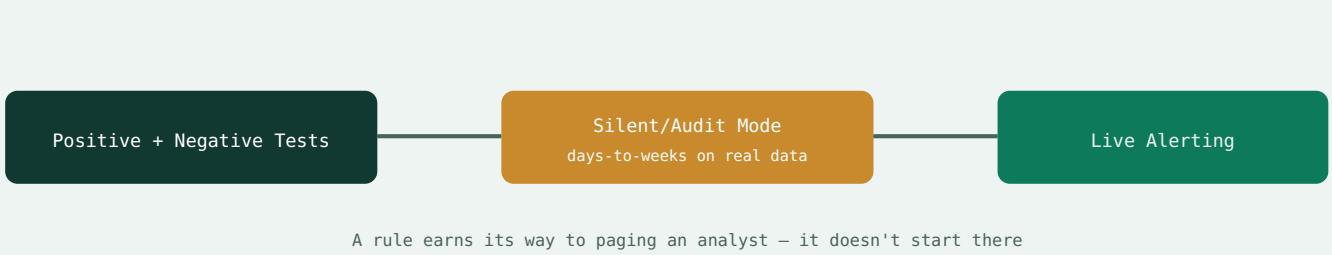

Fig 4.1 — Staged rollout: even a well-tested rule runs silently against real data before it's trusted to actually page anyone.

// SECTION 5

Detection-as-Code

5.1 THE SAME IDEA AS INFRASTRUCTURE AS CODE, APPLIED TO RULES

Detection-as-code stores detection logic — Sigma rules, YARA rules, correlation logic — as version-controlled text, reviewed and deployed through the same pipeline discipline as application software, directly mirroring the Infrastructure as Code principles covered in the SDN & Network Automation guide, Section 4, just applied to detections instead of network configuration.

5.2 WHAT THIS ACTUALLY BUYS YOU

BENEFIT	CONCRETELY
Change history	Every tuning adjustment (SIEM & Log Analysis guide, Section 7) is a tracked commit, not a silent, undocumented edit in a SIEM's web UI
Peer review	A new detection is reviewed before merging — catching logic errors and untested assumptions before Section 4's testing even begins
Repeatability	The exact same detection can be deployed identically across dev, staging, and production SIEM instances
Rollback	A detection causing unexpected noise can be reverted to its last known-good version in seconds, not manually reconstructed from memory

5.3 A TYPICAL PIPELINE

```
$ git commit -m "Add detection for T1003.001 LSASS access"
$ git push origin feature/lsass-detection
$ # → opens a pull request for review
$ # → automated tests run (Section 4.2's positive/negative tests)
$ # → on merge, CI/CD deploys to staging in audit mode (Section 4.3)
$ # → after a defined observation period, promoted to production
```

This pipeline is what actually operationalizes Section 2's lifecycle diagram — Develop and Test aren't separate manual efforts each time, they're stages a pull request automatically moves through, exactly the plan-then-apply discipline the SDN & Network Automation guide's Section 6.2 covers for infrastructure changes, applied here to detection changes instead.

> **Sigma's format-agnostic design (Malware Analysis Basics guide, Section 6.3) is what makes detection-as-code practical across platforms.** A detection stored as Sigma in version control can be converted to whichever SIEM's native format is actually deployed at build time — meaning the same reviewed, tested source of truth works regardless of which specific platform runs it.

// SECTION 6

Measuring Detection Efficacy

6.1 "IT FIRES SOMETIMES" ISN'T A METRIC

The SIEM & Log Analysis guide's Section 7.3 recommended tracking false positive rate "as a real, ongoing metric, not by feel" — this section is the practical detail of what that actually looks like, and why a single metric alone is never the whole picture.

6.2 CORE METRICS

METRIC	MEASURES	WATCH OUT FOR
True Positive Rate	Of all alerts fired, how many were genuinely malicious	A high rate alone doesn't mean the rule is *useful* — see Section 6.3
False Positive Rate	Of all alerts fired, how many were benign	Directly drives alert fatigue risk (SIEM & Log Analysis guide, Section 7.1)
Mean Time to Detect (MTTD)	How quickly a real incident was actually flagged after it began	A rule that's accurate but slow may miss the window where containment is still cheap
Coverage	What fraction of relevant ATT&CK techniques have any detection at all	Covered in depth in Section 7

6.3 VOLUME AND VALUE AREN'T THE SAME THING

A rule that fires constantly on genuinely malicious-but-routine activity (say, an authorized vulnerability scanner triggering a real, technically-accurate detection every single day) has a perfect true-positive rate and zero practical value — analysts learn to ignore it exactly as if it were a poorly-tuned false-positive generator. Efficacy measurement needs to ask not just "was this alert correct" but "did anyone actually need to act on it."

> **Retiring a detection is a legitimate outcome, not a failure.** A rule that consistently measures poorly — low true positive rate, no meaningful MTTD improvement, or alerting on activity nobody ever acts on — should be retired or substantially reworked (looping back to Section 2's lifecycle), rather than left running indefinitely simply because removing it feels like admitting it was a mistake to write in the first place.

// SECTION 7

Coverage Mapping Against ATT&CK

7.1 SEEING DETECTION COVERAGE AS A WHOLE, NOT RULE BY RULE

Individual detections are easy to reason about one at a time; understanding whether an entire environment has *meaningful* coverage requires stepping back and mapping every existing detection against the ATT&CK matrix (Threat Intelligence Fundamentals guide, Section 4.3) — turning a list of rules into a visual map of what's covered and, more importantly, what isn't.

7.2 BUILDING A COVERAGE MAP

STEP	PRODUCES
Tag every existing detection with its ATT&CK technique ID(s)	A direct mapping from rule to technique
Overlay onto the full ATT&CK matrix	A visual heat map — dense in some tactics, empty in others
Cross-reference against relevant threat intelligence	Which uncovered techniques are actually likely to be used against this specific environment (Threat Intelligence Fundamentals guide, Section 1.2's operational tier)
Prioritize gaps by likelihood and impact	A ranked backlog feeding back into Section 2's Identify stage

7.3 DEPTH MATTERS AS MUCH AS BREADTH

A single, narrow rule technically "covering" a technique isn't the same as robust coverage — a technique with one brittle, hash-based detection (bottom of the Pyramid of Pain, Threat Intelligence Fundamentals guide Section 3) sitting at the same coverage-map status as one with multiple behavior-based detections across different data sources creates a false sense of security if the map only tracks yes/no coverage rather than the quality and durability of what's actually there.

> **A coverage map is a living artifact, not a one-time audit.** New techniques get documented in ATT&CK, the environment itself changes, and old detections get retired (Section 6.3) — a coverage map built once and never revisited quickly becomes as unreliable as the individual stale detections it was meant to help identify.

// SECTION 8

Worked Example: Building & Testing a Detection

A coverage-mapping exercise (Section 7) reveals a gap: no detection exists for T1053.005 (Scheduled Task persistence), despite it being flagged as relevant in recent threat intelligence. Walking the full lifecycle from Section 2:

1. **Identify (Section 2.2, 7.3):** the technique is confirmed as a real gap, prioritized given intelligence suggesting it's actively used against similar organizations.
2. **Develop (Section 3):** a Sigma rule is written targeting Event ID 4698 (Windows Event Log Reference guide, Section 5.2), with specific conditions — a task registered to run a script from a non-standard directory — rather than matching on 4698 alone.
3. **Test — positive (Section 4.2):** a scheduled task matching the pattern is created in a safe test environment; the rule fires correctly.
4. **Test — negative (Section 4.2):** the rule is run against two weeks of real historical 4698 events from legitimate software installers and IT-deployed tasks; zero unexpected matches.
5. **Detection-as-code (Section 5):** the rule is committed, reviewed via pull request, and merged.
6. **Staged rollout (Section 4.3):** deployed in silent/audit mode for two weeks against full production volume — zero matches, consistent with the negative test, building confidence before going live.
7. **Deploy live, then measure (Section 6):** promoted to full alerting. One month later: zero false positives, and it has genuinely never fired — a good sign for a well-targeted, low-noise rule, not evidence it's useless.
8. **Update the coverage map (Section 7.2):** T1053.005 is now marked covered, closing the loop the exercise started with.

> **Notice this rule never actually caught anything — and that's fine.** Section 6.3's point about volume versus value cuts both directions: a well-targeted, low-noise rule sitting quietly ready for the technique it was built for is doing its job, even with zero alerts to show for it. The measurement that matters here isn't alert count, it's confirmed absence of both false positives and any gap in the coverage map.

// SECTION 9

Quick Reference & Glossary

TERM	DEFINITION
Detection Engineering	The discipline of building, testing, deploying, and measuring detections systematically.
Positive Testing	Confirming a detection actually fires on the behavior it's meant to catch.
Negative Testing	Confirming a detection doesn't fire on similar, legitimate activity.
Silent/Audit Mode	Running a detection against real data without alerting, to observe behavior before going live.
Detection-as-Code	Storing and managing detection logic as version-controlled, reviewed text.
True Positive Rate	The proportion of alerts that were genuinely malicious.
False Positive Rate	The proportion of alerts that were benign.
Mean Time to Detect (MTTD)	How long after an incident began it was actually flagged.
Coverage Map	A visualization of which ATT&CK techniques have detection coverage, and which don't.

> **End of guide.** Nothing in this guide is about writing smarter rule syntax — the SIEM & Log Analysis and Malware Analysis Basics guides already cover that. This one is about the discipline surrounding the rule: knowing it's tested, knowing it's still worth keeping, and knowing exactly where the gaps still are.